Critique DPO

Zihe Liu, Peter Hayes

February 28, 2026

Abstract

This is *very much* a work-in-progress document, which reviews how generative verifiers can be implemented and simple experimental results. Much of this current version are disjointed concepts and messy; my current task is to tidy these reviews up and include the experimental results.

1 Introduction

As large language models (LLMs) become more capable, fine-tuning techniques to align with human intent are increasingly important. A key consideration for aligning these models is how to most effectively use human resources, or model resources in the case where LLMs themselves are used as oracles. Reinforcement learning from Human or AI preferences (RLHF/RLAIF) is the most prominent example of such a technique but is complex and often unstable. Direct Preference Optimization (DPO) has recently been proposed as a simpler and more stable alternative. In this note, we explore how to expand the DPO objective to make better use of feedback data from the human or AI oracles. Specifically, we incorporate the Chain-Of-Thought reasoning of a generative verifier (acting to verify and critique the validity of base models) in the DPO loss term for the base model.

Asking these oracles to provide reasoning for their binary preferences is common practice. In particular, for LLM-as-a-judge style workflows asking the model to provide reasoning (or so-called chain-of-thought) has been shown to actually improve the model predictions, or verifications in the case of generative verifiers. How can we leverage this rationale when fine-tuning our target model to improve performance in a more data-efficient manner?

2 Background: DPO

For a given prompt x, two completions are sampled from the policy $(y_w, y_l) \sim \pi_\theta(y|x)$ and an oracle judges which it prefers. We denote y_w as the preferred completion and y_l as the other. This process is repeated over N prompts resulting in the pairwise preference dataset $\mathcal{D} = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$.

2.1 Bradley-Terry Preference Model

We can model the judge preferences using the Bradley-Terry (BT) model, which expresses the probability that completion y_1 is preferred over y_2 under a latent reward function r^*:

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(r^*(x, y_1)\right)}{\exp\left(r^*(x, y_1)\right) + \exp\left(r^*(x, y_2)\right)} = \sigma(r^*(x, y_1) - r^*(x, y_2))$$

A key property is that BT depends only on the *difference* of rewards, not their absolute values.

2.2 Standard RLHF

The standard RLHF pipeline comprises 2 stages. First, we learn a reward model r_ϕ where we find the maximum likelihood estimate (MLE) of ϕ under our preference dataset $\mathcal{D}$ and BT model - termed reward modelling. Then, we use RL (PPO) to learn an optimal policy π_θ for this learned reward model.

2.2.1 Reward Modelling

Since r^* is unknown, we learn a parametric approximation r_ϕ. Under the BT model, the likelihood of the observed preference dataset is $\ell(\mathcal{D} \mid \phi) = \prod_{i=1}^{N} \sigma\Big(r_\phi(x^{(i)}, y_w^{(i)}) - r_\phi(x^{(i)}, y_l^{(i)})\Big)$. Taking the log and writing as an expectation, the MLE is obtained by minimising

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{1}$$

2.2.2 RL Fine-Tuning

The learned reward model is then used to fine-tune the policy. The aim is to maximise the KL-constrained objective

$$\max_{\pi_\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(y|x)}\left[r_\phi(x, y)\right] - \beta\, D_{\text{KL}}(\pi_\theta(y|x) \| \pi_{\text{ref}}(y|x)) \tag{2}$$

where π_{ref} is the reference policy (typically the SFT model), kept frozen. The KL term prevents the policy from deviating too far from the distribution on which the reward model is accurate, and $\beta > 0$ controls this trade-off. In practice this is optimised with PPO, requiring four models in memory: the policy, reference, reward model, and value function.

2.3 DPO

DPO [Rafailov et al., 2024] bypasses the reward modelling and RL steps entirely through a change of variables. The key insight is that there exists a bijection between reward functions and their corresponding optimal policies under the KL-constrained objective (2), and this bijection can be exploited to reparameterise the preference likelihood directly in terms of the policy. We follow the expositions by [Wolfe, 2025] and [Romero, 2024] in our presentation of the derivation below.

2.3.1 Optimal Policy

For *any* reward function r (not just the learned r_ϕ or the latent r^*), the KL-constrained objective (2) can be rewritten as a minimisation of a KL divergence between π and a target distribution defined by r, plus a constant. Specifically, letting $Z_r(x) = \sum_y \pi_{\text{ref}}(y|x) \exp\Big(\frac{1}{\beta} r(x, y)\Big)$ be a partition function that depends on x and r but is constant with respect to π, the objective becomes

$$\min_{\pi} \; \mathbb{E}_x \left[D_{\text{KL}}\left(\pi(y|x) \;\Big\|\; \frac{1}{Z_r(x)}\, \pi_{\text{ref}}(y|x)\, \exp\Big(\tfrac{1}{\beta}\, r(x, y)\Big)\right) - \log Z_r(x) \right]$$

By Gibbs' inequality ($D_{\text{KL}} \geq 0$, with equality iff the distributions are identical), the unique optimum is

$$\pi_r(y|x) = \frac{1}{Z_r(x)}\, \pi_{\text{ref}}(y|x)\, \exp\left(\frac{1}{\beta}\, r(x, y)\right) \tag{3}$$

This defines a bijection: every reward r maps to a unique optimal policy π_r, and conversely every policy π_r defines a unique reward[1] r for which it is optimal. Using the latent reward r^* gives the optimal policy $\pi^* \equiv \pi_{r^*}$ for our latent rewards.

2.3.2 Reparameterisation

Rearranging (3) for the reward expresses any reward purely in terms of its corresponding optimal policy and a prompt-dependent partition function:

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z_r(x)$$

[1] We really mean unique equivalence class of rewards and its optimal policy; see Lemma 1 and Lemma 2 in [Rafailov et al., 2024].

Substituting the latent reward $r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z_{r^*}(x)$ into the BT model, the $\beta \log Z_{r^*}(x)$ terms cancel because BT depends only on reward *differences* and $Z_{r^*}(x)$ is independent of y:

$$p^*(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

The preference model now has no dependence on a reward model — it is expressed entirely in terms of policies.

2.3.3 DPO Loss

We replace the fixed π^* with a learnable π_θ and maximise the likelihood of the observed preferences, conditioned on our policy parameters θ. Defining the *implicit reward*

$$\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \tag{4}$$

the DPO loss is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)\right)\right] \tag{5}$$

This is identical in form to the reward model loss (1), but conditioned on the policy parameters θ instead of the reward model parameters ϕ. Traditionally, RLHF performs MLE on $\mathcal{D}$ conditioned on ϕ to learn a reward model, then uses RL to find the optimal policy for that reward. DPO instead performs MLE on $\mathcal{D}$ conditioned directly on θ. Because of the reward–policy bijection, improving π_θ to better fit the preference data simultaneously yields an implicit reward $\hat{r}_\theta$ for which π_θ is guaranteed to be optimal. This eliminates both the reward modelling step and the RL step, requiring only two models in memory (the policy π_θ and the frozen reference π_{ref}).

In this work we suggest simple extensions to this objective.

3 Literature Review

We review research incorporating reasoning and critique data into fine-tuning, building toward the critique-augmented preference optimisation approach of this work. We adopt a unified notation: π_θ denotes the learnable policy, π_{ref} the frozen reference (usually the base SFT model), x the prompt, y_w and y_l the preferred and rejected responses, c a model's own chain-of-thought reasoning trace, r an oracle or verifier critique rationale, $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ the implicit reward.

The review is organised around a central question: *what kind of signal is being used, and for which model?* The narrative moves from binary AI feedback (Section 3.1), through enriching DPO with supervised reasoning targets (Section 3.2), to methods that improve verifiers by having them reason (Section 3.4), and finally to the closest work — methods that bring verifier critique directly into the base model's training objective (Section 3.5).

3.1 Beyond Binary Feedback

3.1.1 RLAIF and the Information Bottleneck

RL from AI Feedback (RLAIF) replaces human annotators with LLM oracles. [Bai et al., 2022] first use LLM-generated labels for harmlessness finetuning, doing RL with a preference model trained with LLM preferences. [Lee et al., 2024] use LLM to directly give rewards, showing that RLAIF matches or exceeds RLHF on summarisation and dialogue, even when the labeller is the same model being aligned. [Li et al., 2024] instruct the labeller to reason explicitly and report modest improvements. A shared limitation is that the oracle's reasoning is discarded after labelling: the sole training signal is a binary or scalar preference label, creating an information bottleneck that the remainder of this review seeks to address.

3.1.2 Impact of Chain-of-Thought Data for DPO

[Liu et al., 2025] provide a systematic empirical study of CoT's impact on DPO. In text-to-SQL — where preference datasets traditionally contain only final SQL queries *without* reasoning steps — DPO degrades performance across the majority of ten tested LLMs. Augmenting the same datasets with synthetically generated chain-of-thought solutions achieves consistent DPO improvements for the first time. The analysis identifies three mechanisms. First, *reward hacking mitigation*: without CoT, the implicit reward (4) can be inflated by superficial correlates (longer SQL, non-executable patterns) that do not reflect reasoning quality. Model performance initially peaks then decreases, while the implicit reward continues to rise. With CoT, the model's self-assessed implicit reward tracks actual performance. Second, *discriminative calibration*: CoT-trained models maintain stable classification accuracy between correct and incorrect solutions throughout training, whereas vanilla DPO becomes unstable. Third, *sample efficiency*: CoT-augmented DPO continues to improve with additional data, while vanilla DPO saturates. 2 samples of CoT with DPO achieves the same performance gains as 16 samples of vanilla DPO.

This finding directly motivates our approach: if the model's own CoT improves DPO, then an external verifier's critique — which provides a richer perspective on why one response is preferred — should be informative too.

3.1.3 Self-Taught Reasoning: STaR and V-STaR

STaR [Zelikman et al., 2022] introduced the template of iterative self-training on CoT data: sample reasoning traces from the current model, filter for those yielding correct answers, and fine-tune. V-STaR [Hosseini et al., 2024] extend this by simultaneously training a verifier via DPO on correct versus incorrect solutions, enabling the generator and verifier to co-improve. Both methods establish that CoT data, even when self-generated, is a substantially richer training signal than outcome labels alone.

3.2 DPO + NLL: Anchoring Preferences with Supervised Targets

A recurring structural pattern in the literature combines the DPO preference loss with an auxiliary negative log-likelihood (NLL) term on a supervised target. We present the generic template first, then show how each paper instantiates it with a **different NLL target** t:

$$\mathcal{L}_{\text{DPO+NLL}} = \underbrace{-\log \sigma(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l))}_{\text{preference term}} + \alpha \underbrace{\mathcal{L}_{\text{NLL}}(t \mid x)}_{\text{supervised anchor}} \tag{6}$$

The DPO term provides a contrastive signal (push apart y_w and y_l), while the NLL term anchors the policy to a specific supervised target. As we show below, the *choice of* t is what distinguishes these methods from one another — and from Method 1 of this paper.

3.2.1 Iterative Reasoning Preference Optimization (IRPO)

[Pang et al., 2024] set the NLL target to the **model's own winning chain-of-thought**: $t = (c_w, y_w)$. At each iteration, the model samples N reasoning traces per prompt, partitions them into correct (c_w, y_w) and incorrect (c_l, y_l) by checking the final answer against ground truth, and optimises:

$$\mathcal{L}_{\text{IRPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(c_w, y_w \mid x)}{\pi_{\text{ref}}(c_w, y_w \mid x)} - \beta \log \frac{\pi_\theta(c_l, y_l \mid x)}{\pi_{\text{ref}}(c_l, y_l \mid x)}\right) - \alpha \frac{\log \pi_\theta(c_w, y_w \mid x)}{|c_w| + |y_w|} \tag{7}$$

The NLL term is length-normalised by $|c_w| + |y_w|$ to avoid a bias toward shorter sequences. Crucially, the reference model π_{ref} is updated to π_θ at each iteration before re-sampling.

Why the NLL term is necessary. Under standard DPO (i.e. $\alpha = 0$), the log-probabilities of *chosen* sequences actually *decrease* during training — the model achieves the required preference margin by pushing rejected probabilities down faster than chosen ones fall. The NLL anchor prevents this collapse: with $\alpha > 0$, chosen log-probabilities increase while rejected ones decrease, creating a healthy expanding margin. Empirically, DPO alone achieves 61.8% on GSM8K while DPO+NLL achieves 73.1% at iteration 1; after four iterations, IRPO reaches **81.6%** (88.7% with majority voting), starting from Llama-2-70B-Chat at 55.6%.

Connection to Method 1. The loss in (7) is structurally identical to Method 1's objective. The key difference is the *NLL target*: IRPO's $t = (c_w, y_w)$ is the model's own correct reasoning, whereas Method 1's $t = r$ is an *external verifier's critique* explaining why $y_w \succ y_l$. Method 1 can therefore be understood as IRPO where the NLL target is replaced with the oracle's preference rationale.

3.2.2 AIPO: Agreement-Aware Iterative Preference Optimization

[Shen et al., 2024] set the NLL target to the **winning response**: $t = y_w$. Their key modification is to the DPO term itself. Defining the log-probability ratios

$$s_\theta = \log \frac{\pi_\theta(y_w \mid x)}{\pi_\theta(y_l \mid x)}, \qquad s_{\text{ref}} = \log \frac{\pi_{\text{ref}}(y_w \mid x)}{\pi_{\text{ref}}(y_l \mid x)},$$

standard DPO is $-\log \sigma(\beta(s_\theta - s_{\text{ref}}))$. AIPO introduces an agreement-aware coefficient $\alpha > 0$:

$$\mathcal{L}_{\alpha\text{-DPO}} = -\mathbb{E}_{\mathcal{D}}\big[\log \sigma\big(\beta\big(s_\theta - (1+\alpha)\, s_{\text{ref}}\big)\big)\big] \tag{8}$$

The gradient weight decomposes as $\sigma(\beta(s_{\text{ref}} - s_\theta + \alpha\, s_{\text{ref}}))$: the additional $\alpha\, s_{\text{ref}}$ amplifies the learning signal when the reference model agrees with the preference label and dampens it when there is disagreement, mitigating the length exploitation that compounds across iterative DPO. The full AIPO loss combines this with a length-normalised NLL term:

$$\mathcal{L}_{\text{AIPO}} = \mathcal{L}_{\alpha\text{-DPO}} + \lambda \cdot \mathcal{L}_{\text{NLL}}, \qquad \mathcal{L}_{\text{NLL}} = -\frac{1}{|y_w|} \log \pi_\theta(y_w \mid x) \tag{9}$$

AIPO achieves 57.1% length-controlled win rate on AlpacaEval 2.0 with Gemma-2-9B-It and 81.6% on Arena-Hard with Mistral-Large, outperforming GPT-4.

3.2.3 Direct Judgement Preference Optimization (DJPO)

[P. Wang et al., 2024] set the NLL target to the **verifier's CoT critique and judgment**: $t = (c, j)$, where c is a chain-of-thought critique and j is the final judgment. Crucially, the model being trained is a **judge**, not the base policy. They construct three types of DPO preference pairs — CoT critique $\mathcal{D}_{\text{CoT}}$, standard judgment $\mathcal{D}_{\text{Std}}$, and response deduction $\mathcal{D}_{\text{Ded}}$ — and train with:

$$\mathcal{L}_{\text{DJPO}} = -\frac{\log \pi_\theta(y_w \mid x)}{|y_w| + |x|} - \log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right) \tag{10}$$

where y_w and y_l are correct and incorrect judge evaluations respectively. Using 680K preference pairs with a 70:15:15 ratio across the three types, they train SFR-Judge models at 8B, 12B, and 70B scales. SFR-Judge-8B outperforms GPT-4o on aggregate pairwise evaluation across 7 benchmarks; SFR-Judge-70B achieves the best overall score of 84.25%.

Connection to Method 1. DJPO has the same DPO + NLL-on-critique structure as Method 1. The distinction is feature (C): DJPO trains a standalone *judge model*, whereas Method 1 applies the same objective to the *base policy model* that generates answers.

3.2.4 Summary: The NLL Target Determines the Method

Method	NLL target t	Target model	Key result
IRPO	Model's own CoT c_w	Base policy	GSM8K 81.6%
AIPO	Winning response y_w	Base policy	AlpacaEval LC 57.1%
DJPO	Verifier critique (c, j)	Judge model	Outperforms GPT-4o
Method 1	**Oracle critique** r	**Base policy**	—

Method 1 uniquely combines the verifier's critique as the NLL target with training the base policy model — occupying the cell that no prior method fills.

3.3 Critique Fine-Tuning: NLL on Critique Without DPO

[Y. Wang et al., 2025] ask: what happens if we use *only* the NLL term on critique data, without any preference optimisation? In Critique Fine-Tuning (CFT), the base model is trained to critique a given query-response pair by maximising:

$$\mathcal{L}_{\text{CFT}}(\theta) = -\sum_{t=1}^{|c|} \log \pi_\theta(c_t \mid c_{<t},\, [x;\, y]) \tag{11}$$

where $[x;\, y]$ concatenates the query and a (potentially flawed) response, and c is a critique generated by a teacher model (GPT-4o). At inference, the CFT-trained model generates *direct answers* without producing critiques — the critique mechanism shapes internal reasoning only during training.

Using just 50K examples and 8 H100 GPU-hours, CFT yields 4–10% improvement over SFT across six mathematical benchmarks. Qwen2.5-Math-7B-CFT matches models trained on 2M+ samples and rivals RL-based methods using 140× more compute. Notably, combining CFT with SFT data *hurts* performance (54.8% vs. 57.1%), suggesting interference between critique and imitation objectives.

Connection to Method 1. CFT is exactly the second term of Method 1's objective in isolation: it applies NLL on critique reasoning to the base model (features B and C), but without a DPO preference term (feature A). Method 1 can be understood as CFT augmented with DPO to simultaneously encourage the model to prefer better answers.

3.4 Generative Verifiers and Critique-Augmented Reward Models

The methods above train the *policy* model. A parallel line of work improves the *verifiers and reward models* that judge responses, by having them *generate* reasoning before scoring. These methods establish that critique data is a powerful training signal for the judging side — and thereby produce the very data (the verifier's rationale r) that Method 1 uses as an NLL target for the policy.

3.4.1 Generative Verifiers (GenRM)

[L. Zhang et al., 2024] recast verification as next-token prediction rather than scalar classification. In the simplest variant (**GenRM-Direct**), the model predicts whether a solution is correct via a single token, trained on:

$$\mathcal{D}_{\text{Direct}} = \{(x, y^+, I),\ \text{'Yes'}\} \cup \{(x, y^-, I),\ \text{'No'}\}$$

where $I =$ "Is the answer correct (Yes/No)?", and the verification score at inference is simply $\pi_\theta(\text{'Yes'} \mid x, y, I)$.

GenRM-CoT extends this by generating a verification chain-of-thought v_{CoT} before the Yes/No token. A unified loss jointly trains on verification and solution generation:

$$\mathcal{L}_{\text{GenRM}} = \mathcal{L}_{\text{SFT}}(\theta,\, \mathcal{D}_{\text{verify}}) + \lambda\, \mathcal{L}_{\text{SFT}}(\theta,\, \mathcal{D}_{\text{correct}}) \tag{12}$$

where $\mathcal{D}_{\text{correct}}$ is a set of correct solutions used for generation training, and $\lambda > 0$ controls the mixture. At inference, multiple CoT rationales can be sampled and their verdicts aggregated via majority voting: $r_{\text{MV@}K} = \frac{1}{K}\sum_{i=1}^{K} \pi_\theta(\text{'Yes'} \mid x, y, v_{\text{CoT}}^{(i)})$.

GenRM-CoT outperforms discriminative reward models, DPO verifiers, and LLM-as-a-Judge, with gains ranging from 5% to 45.3% on algorithmic tasks and 73% to 93.4% on GSM8K. The CoT verification rationale v_{CoT} is precisely the oracle reasoning r that Method 1 uses as an NLL training target for the base policy.

3.4.2 GenRM-STaR-DPO: Training Judges with DPO on Rationales

[Mahan et al., 2024] apply DPO to the judge's own self-generated rationales, creating preference pairs from correct and incorrect reasoning traces. The STaR-DPO loss is:

$$\mathcal{L}_{\text{GenRM-DPO}} = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\phi(I_w, r_w \mid x, y_1, y_2)}{\pi_{\text{ref}}(I_w, r_w \mid x, y_1, y_2)} - \beta \log \frac{\pi_\phi(I_l, r_l \mid x, y_1, y_2)}{\pi_{\text{ref}}(I_l, r_l \mid x, y_1, y_2)}\right)\right] \tag{13}$$

where r_w are rationales leading to the correct judgment I_w and r_l lead to incorrect judgments I_l. Through iterative STaR (3 iterations), the model self-bootstraps improving rationales. STaR-DPO matches Bradley-Terry reward models in-distribution while outperforming them by 10–45% out-of-distribution on RewardBench. Critically, self-generated rationales outperform off-policy rationales from GPT-4, which cause distribution mismatch.

This loss is equivalent in form to Method 1, but applied to train a standalone *judge* rather than the base policy.

3.4.3 Critique-out-Loud Reward Models (CLoud)

[Ankner et al., 2024] train reward models that generate a natural-language critique c before predicting a scalar reward $r_{\theta_R}(x, y, c)$, via a three-stage pipeline. First, the base model is SFT'd on oracle critiques (from Llama-3.1-405B). Second, oracle critiques are *replaced* with self-generated ones to form $\mathcal{D}_{\text{self}}$. Third, the model is trained on the joint loss:

$$\mathcal{L}_{\text{CLoud}} = \underbrace{-\mathbb{E}[\log \sigma(r_{\theta_R}(x, y_w, c_w) - r_{\theta_R}(x, y_l, c_l))]}_{\text{reward modelling}} + \lambda \underbrace{\mathcal{L}_{\text{SFT}}(\theta_B, \theta_{LM}, \mathcal{D})}_{\text{critique generation}} \tag{14}$$

Key insight: on-policy vs. off-policy critiques. Training the reward head on oracle (off-policy) critiques causes a 5.6 percentage point drop in RewardBench accuracy for the 8B model compared to self-generated (on-policy) critiques, due to distribution shift between training and inference. This observation has direct implications for Method 1: the base model's NLL target r should ideally be drawn from a distribution the model can learn to approximate at inference.

CLoud-8B improves pairwise classification accuracy by 4.65 points over classic reward models and achieves Pareto-dominant Best-of-N win rates on ArenaHard.

3.4.4 Critic-RM: Critiques as Latent Variables

[Yu et al., 2025] provide an elegant variational formulation that treats critiques as latent variables. Denoting z_w, z_l as critiques for chosen and rejected responses, the preference likelihood is marginalised:

$$\log p(y_w \succ y_l \mid x) \;\geq\; \mathbb{E}_{q_\phi(z_w|y_w,x),\, q_\phi(z_l|y_l,x)} \left[\log \frac{p(y_w \succ y_l,\, z_w, z_l \mid x)}{q_\phi(z_w|y_w, x)\, q_\phi(z_l|y_l, x)} \right] \tag{15}$$

This variational lower bound decomposes into two objectives trained jointly: (1) a **preference modeling loss** $\ell_r = -\log \sigma(r_\psi(x, [y_w; z_w]) - r_\psi(x, [y_l; z_l]))$, where $[y; z]$ concatenates response and critique; and (2) a **critique generation loss** $\ell_c = -\frac{1}{K} \sum_{z \in Z} \log q_\phi(z \mid y, x)$, which trains the critique model to approximate the oracle distribution. A dynamic weight schedule $\lambda(t)$ emphasises critique generation in early epochs and gradually shifts to reward prediction.

Critic-RM improves reward modeling accuracy by 3.7–7.3% on RewardBench. The 70B variant outperforms both GPT-4 and Llama-3.1-405B on critique quality, and matches full-data performance with only 10% of training data.

3.4.5 Outcome-Process Inconsistency: RM-NLHF

[Z. Wang et al., 2026] identify a fundamental failure mode of outcome-only RL for generative reward models: **outcome-process inconsistency**, where a GRM correctly predicts the preferred response but generates a logically flawed critique to justify the prediction. Across frontier models, 20–33% of correct predictions are accompanied by unsound reasoning (including 26% for Gemini-2.5-Pro and 33.6% for Claude-3.7-Sonnet).

To address this, RM-NLHF adds a *process reward* R_{process} based on the F1 similarity between the GRM's critique and a human oracle critique. The composite training reward is:

$$R(x, y_w, y_l) = \begin{cases} -1 & \text{invalid format} \\ 0 & \text{incorrect outcome label} \\ 1 + \lambda \cdot R_{\text{process}} & \text{correct outcome label} \end{cases} \tag{16}$$

Since human critiques are expensive, a **MetaRM** — a small regression model trained on a seed human-critique dataset — predicts R_{process} for unlabelled data. An online variant co-evolves with GRM training to handle distribution shift.

RM-NLHF-Qwen-7B achieves 0.648 overall versus 0.576 for the outcome-only baseline. When used as the reward model in a downstream RL pipeline, it improves policy performance by 12.9% on Arena-Hard-V2.0. This paper provides perhaps the strongest empirical argument for the core premise of Critique DPO: supervising the *quality* of verifier reasoning — not just outcome correctness — is essential for robust alignment.

3.4.6 Process Reward Models: ThinkPRM

[Khalifa et al., 2025] extend the generative verifier idea to process-level rewards. ThinkPRM generates a verification chain-of-thought *before* scoring each reasoning step, trained via standard NLL on ∼1,000 filtered synthetic verification traces (∼8K step-level labels — roughly 1% of PRM800K's 712K labels [Lightman et al., 2024]). Despite this extreme data efficiency, ThinkPRM surpasses discriminative PRMs trained on the full PRM800K dataset by 8% on GPQA-Diamond and 4.5% on LiveCodeBench. [Zhao et al., 2025] extend this further with code-augmented verification, where a 7B GenPRM exceeds Qwen2.5-Math-PRM-72B using only 23K examples. Empirical evidence from Kimi k1.5 [Kimi Team, 2025] — where a CoT reward model achieves 98.5% annotation accuracy versus 84.4% for a scalar model — provides additional motivation for using verifier reasoning as a training signal.

These results validate a core premise of this paper: generative verifier reasoning is not merely a byproduct to discard, but a signal of genuine value that yields substantial performance gains when used during training.

3.5 Training the Base Model with Critique

The methods in Section 3.4 improve reward models and judges by having them generate critique. A closely related cluster instead trains the base *policy* model directly using critique signals. This section covers the methods most directly related to Method 1.

Recall the three defining features of Method 1: **(A)** a DPO loss on answer preferences (y_w, y_l); **(B)** an NLL term on the oracle/verifier's CoT rationale r; and **(C)** applied to the base policy model. No single prior paper combines all three; the methods below each match two of the three. (CFT, discussed in Section 3.3, provides B+C without A.)

3.5.1 SuperCorrect: Cross-Model Collaborative DPO

[Yang et al., 2024] introduce a two-stage framework. Stage 1 distils hierarchical thought templates from a large teacher model (o1-preview) via SFT. Stage 2 applies **cross-model collaborative DPO** on teacher-guided error-correction traces. Given an erroneous student solution with the first error at step k, the teacher generates a correction trace (a_k^+, c_k^+) (error analysis and correction guidance) while the student independently generates (a_k^-, c_k^-). The DPO loss is:

$$\mathcal{L}_{\text{Cross-DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(a_k^+, c_k^+ \mid x, s_{<k})}{\pi_{\text{ref}}(a_k^+, c_k^+ \mid x, s_{<k})} - \beta \log \frac{\pi_\theta(a_k^-, c_k^- \mid x, s_{<k})}{\pi_{\text{ref}}(a_k^-, c_k^- \mid x, s_{<k})}\right)\right] \tag{17}$$

where $s_{<k}$ are the correct preceding steps. The teacher's correction trace plays the role of r in Method 1. SuperCorrect-7B achieves 70.2% on MATH (surpassing DeepSeekMath-7B by 7.8%) and demonstrates that cross-model DPO at the step level substantially improves self-correction capabilities. CriticGPT [McAleese et al., 2024] and CTRL [Xie et al., 2025] provide complementary evidence that LLMs can be explicitly taught to generate high-quality critiques, validating the practical feasibility of constructing the oracle critique data r used in Method 1.

3.5.2 Critique-GRPO: The Closest RL Analogue

[X. Zhang et al., 2025] is the closest existing paper to Method 1 in spirit. Critique-GRPO integrates natural-language critiques with RL policy optimisation, training the base model to learn from both initial

responses and critique-guided refinements. The combined objective is:

$$J_{\text{C-GRPO}} = \frac{1}{n}\sum_{i=1}^{n}\sum_{t} r_t^{(i)}(\theta)\,\hat{A}_t^{(i)} \;+\; \frac{1}{k}\sum_{i'=1}^{k}\sum_{t} f\left(r_{\text{ref},t}^{(i')}(\theta)\right)\hat{A}_t^{(i')} \tag{18}$$

where $r_t^{(i)}(\theta) = \pi_\theta/\pi_{\theta_{\text{old}}}$ is the standard importance sampling ratio and $\hat{A}$ are group-normalised advantages computed over the mixed batch of initial and refined responses. The key technical contribution is a **policy shaping function** for refined responses:

$$f(x) = \frac{\pi_\theta}{\pi_\theta + \gamma}, \qquad 0 < \gamma < 1$$

When π_θ is small (unfamiliar token from critique-guided refinement), $f \approx \pi_\theta/\gamma$ upweights the gradient; when π_θ is large (familiar token), $f \approx 1$. This amplifies learning from correct refinements that the model would not have explored autonomously. Removing the shaping function degrades average pass@1 from 47.08% to 43.95%.

	Method 1	**Critique-GRPO**
Algorithm	DPO (offline)	GRPO (online RL)
Critique source	External oracle	Self-generated
Training regime	Offline preference pairs	Online rollouts

Critique-GRPO achieves +4.4% on Qwen2.5-7B-Base and +16.7% pass@1 on AIME 2024 (self-critiquing). These two methods arrive at the same conceptual combination — critique reasoning and preference optimisation for the base model — from different training paradigms.

3.5.3 iGRPO: Self-Feedback Without Explicit Critique

[Hatamizadeh et al., 2026] extend GRPO with a two-stage self-feedback loop that outperforms Critique-GRPO *without* generating natural-language critiques. In Stage 1, N drafts are sampled and scored; the best draft $\hat{d}$ is selected. In Stage 2, the prompt is augmented with $\hat{d}$ and G refinements are sampled for GRPO optimisation, with the total budget held constant ($N + G = G_{\text{GRPO}}$). The expected reward of the selected draft satisfies:

$$\mathbb{E}[R(\hat{d})] = 1 - (1 - V_\theta(q))^N \tag{19}$$

which is monotonically increasing in the policy's success probability $V_\theta(q)$, creating a bootstrapping dynamic: as the policy improves, its best draft improves, which provides better conditioning for further refinement. iGRPO also delays entropy collapse, maintaining broader exploration.

Results: Nemotron-H-8B-Base (GRPO 41.1% → Critique-GRPO 43.4% → iGRPO **45.0%**); state-of-the-art for 7B models on AIME 2024 (**85.6%**) and AIME 2025 (**79.6%**).

3.5.4 RL Tango: Joint Generator–Verifier Training

[Zha et al., 2025] simultaneously train both a policy generator π_g and a generative process-level verifier π_v using interleaved RL. The generator receives a **combined advantage** blending outcome and step-level signals from the verifier:

$$\hat{A}_{g,t}^{(i)} = (1-\alpha)\,\hat{A}_{g,\text{out},t}^{(i)} + \alpha\,\hat{A}_{g,\text{step},t}^{(i)} \tag{20}$$

where $\hat{A}_{g,\text{step},t}^{(i)} = \sum_{k:I(k)\geq t} \frac{y_{\text{step},k}/K - \mu}{\sigma}$ accumulates normalised verifier judgments $y_{\text{step},k} \in \{-1, 1\}$ from all subsequent steps. The blending weight α **decays exponentially** during training, emphasising step-level guidance early and shifting to outcome signals to mitigate reward hacking. The verifier is trained with a class-aware advantage to handle the imbalance between correct and incorrect generator solutions, and is rewarded solely on outcome correctness — its step-level CoT reasoning *emerges* from outcome-level rewards alone.

Tango matches vanilla GRPO's 200-step accuracy in approximately 60 steps (a 3.3× efficiency improvement) and achieves best-in-class performance among 7B models, with the verifier leading on ProcessBench. The result validates the core premise of Method 1: verifier reasoning is a useful learning signal for the generator, even when the two are trained jointly from outcome rewards alone.

3.6 Positioning of Method 1

The closest prior work to Method 1 divides into three groups that each match two of its three defining features:

Paper	A: DPO on answers	B: NLL on verifier CoT	C: Base model
Method 1 (this paper)	✓	✓	✓
IRPO [Pang et al., 2024]	✓	NLL on own CoT	✓
AIPO [Shen et al., 2024]	✓	NLL on y_w	✓
DJPO [P. Wang et al., 2024]	✓	✓	× (judge)
GenRM-STaR-DPO [Mahan et al., 2024]	✓	✓	× (judge)
CFT [Y. Wang et al., 2025]	×	✓	✓
Critique-GRPO [X. Zhang et al., 2025]	GRPO	Self-critique	✓
SuperCorrect [Yang et al., 2024]	On traces	Teacher correction	✓
RL Tango [Zha et al., 2025]	Via RL	Verifier CoT	Co-training

Method 1 sits at the intersection of three lines of work that have so far only been combined in pairs: (1) IRPO's DPO+NLL loss structure — the right loss form, but the wrong NLL target; (2) DJPO and GenRM-STaR-DPO's use of verifier rationale as the NLL target — the right data, but training the wrong model; and (3) CFT's base-model orientation — the right model, but missing the DPO preference term. Concurrently, Critique-GRPO arrives at a closely analogous idea from the online RL direction, providing independent empirical validation that joint critique-reasoning and preference optimisation is a productive direction for base model training.

References

Rafailov, Rafael et al. (2024). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. arXiv: 2305.18290 [cs.LG]. URL: https://arxiv.org/abs/2305.18290.

Wolfe, Cameron R. (2025). *Direct Preference Optimization (DPO)*. Blog post. URL: https://cameronrwolfe.substack.com/p/direct-preference-optimization.

Romero, Tyler (2024). *Direct Preference Optimization Explained In-depth*. Blog post. URL: https://www.tylerromero.com/posts/2024-04-dpo/.

Bai, Yuntao et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv: 2212.08073 [cs.CL]. URL: https://arxiv.org/abs/2212.08073.

Lee, Harrison et al. (2024). *RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*. arXiv: 2309.00267 [cs.CL]. URL: https://arxiv.org/abs/2309.00267.

Li, Ang et al. (2024). *HRLAIF: Improvements in Helpfulness and Harmlessness in Open-domain Reinforcement Learning From AI Feedback*. arXiv: 2403.08309 [cs.LG]. URL: https://arxiv.org/abs/2403.08309.

Liu, Hanbing et al. (2025). *Uncovering the Impact of Chain-of-Thought Reasoning for Direct Preference Optimization*. arXiv: 2502.11656 [cs.CL]. URL: https://arxiv.org/abs/2502.11656.

Zelikman, Eric et al. (2022). *STaR: Bootstrapping Reasoning With Reasoning*. arXiv: 2203.14465 [cs.LG]. URL: https://arxiv.org/abs/2203.14465.

Hosseini, Arian et al. (2024). *V-STaR: Training Verifiers for Self-Taught Reasoners*. arXiv: 2402.06457 [cs.LG]. URL: https://arxiv.org/abs/2402.06457.

Pang, Richard Yuanzhe et al. (2024). *Iterative Reasoning Preference Optimization*. arXiv: 2404.19733 [cs.CL]. URL: https://arxiv.org/abs/2404.19733.

Shen, Yaojie et al. (2024). *AIPO: Improving Training Objective for Iterative Preference Optimization*. arXiv: 2409.08845 [cs.CL]. URL: https://arxiv.org/abs/2409.08845.

Wang, Peifeng et al. (2024). *Direct Judgement Preference Optimization*. arXiv: 2409.14664 [cs.CL]. URL: https://arxiv.org/abs/2409.14664.

Wang, Yubo, Xiang Yue, and Wenhu Chen (2025). *Critique Fine-Tuning: Learning to Critique is More Effective than Learning to Imitate*. arXiv: 2501.17703 [cs.CL]. URL: https://arxiv.org/abs/2501.17703.

Zhang, Lunjun et al. (2024). *Generative Verifiers: Reward Modeling as Next-Token Prediction*. arXiv: 2408.15240 [cs.LG]. URL: https://arxiv.org/abs/2408.15240.

Mahan, Dakota et al. (2024). *Generative Reward Models*. arXiv: 2410.12832 [cs.LG]. URL: https://arxiv.org/abs/2410.12832.

Ankner, Zachary et al. (2024). *Critique-out-Loud Reward Models*. arXiv: 2408.11791 [cs.LG]. URL: https://arxiv.org/abs/2408.11791.

Yu, Yue et al. (2025). *Self-Generated Critiques Boost Reward Modeling for Language Models*. arXiv: 2411.16646 [cs.CL]. URL: https://arxiv.org/abs/2411.16646.

Wang, Zongqi et al. (2026). *Reward Modeling from Natural Language Human Feedback*. arXiv: 2601.07349 [cs.CL]. URL: https://arxiv.org/abs/2601.07349.

Khalifa, Muhammad et al. (2025). *Process Reward Models That Think*. arXiv: 2504.16828 [cs.LG]. URL: https://arxiv.org/abs/2504.16828.

Lightman, Hunter et al. (2024). "Let's Verify Step by Step". In: *ICLR*. arXiv: 2305.20050 [cs.LG]. URL: https://arxiv.org/abs/2305.20050.

Zhao, Jian et al. (2025). *GenPRM: Scaling Test-Time Compute of Process Reward Models via Generative Reasoning*. arXiv: 2504.00891 [cs.LG]. URL: https://arxiv.org/abs/2504.00891.

Kimi Team (2025). *Kimi k1.5: Scaling Reinforcement Learning with LLMs*. arXiv: 2501.12599 [cs.CL]. URL: https://arxiv.org/abs/2501.12599.

Yang, Ling et al. (2024). *SuperCorrect: Advancing Small LLM Reasoning with Thought Template Distillation and Self-Correction*. arXiv: 2410.09008 [cs.CL]. URL: https://arxiv.org/abs/2410.09008.

McAleese, Nat et al. (2024). *LLM Critics Help Catch LLM Bugs*. arXiv: 2407.00215 [cs.CL]. URL: https://arxiv.org/abs/2407.00215.

Xie, Zhihui et al. (2025). *Teaching Language Models to Critique via Reinforcement Learning*. arXiv: 2502.03492 [cs.CL]. URL: https://arxiv.org/abs/2502.03492.

Zhang, Xiaoying et al. (2025). *Critique-GRPO: Advancing LLM Reasoning with Natural Language and Numerical Feedback*. arXiv: 2506.03106 [cs.CL]. URL: https://arxiv.org/abs/2506.03106.

Hatamizadeh, Ali et al. (2026). *iGRPO: Self-Feedback-Driven LLM Reasoning*. arXiv: 2602.09000 [cs.CL]. URL: https://arxiv.org/abs/2602.09000.

Zha, Kaiwen et al. (2025). *RL Tango: Reinforcing Generator and Verifier Together for Language Reasoning*. arXiv: 2505.15034 [cs.CL]. URL: https://arxiv.org/abs/2505.15034.